Multi-Cluster MongoDB HA/DR on OpenShift: Technical Overview

Purpose

This document describes the architecture, technical decisions, testing outcomes, and deployment prerequisites for a self-hosted MongoDB Enterprise High Availability / Disaster Recovery solution spanning multiple independent OpenShift clusters. The solution uses the MongoDB Controllers for Kubernetes (MCK) 1.8.0 operator with MongoDB Ops Manager and provides automatic cross-site failover with zero manual intervention.

Disclaimer

This proof of concept was built and tested on AWS-hosted OpenShift clusters (OpenShift 4.20) across three AWS regions (us-east-2, ap-southeast-1, eu-central-1). The target production deployment is expected to be on-premises across two corporate data centres with a third arbiter site.

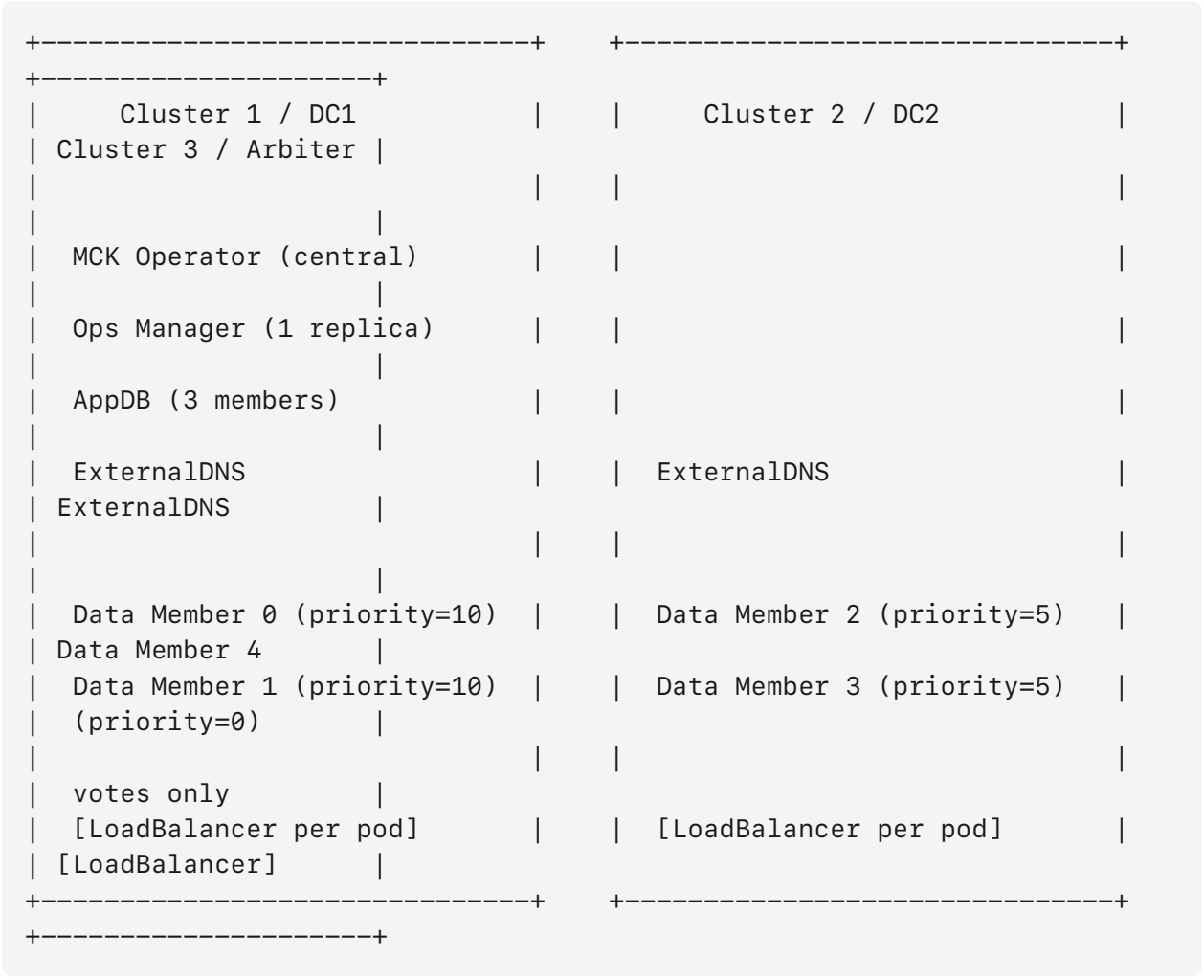
Impact of on-premises deployment:

- **Load balancers:** AWS NLBs used in the PoC would be replaced by MetalLB, F5, or an equivalent on-premises load balancer. The `service.beta.kubernetes.io/aws-load-balancer-type: "nlb"` annotation would change to match the on-prem load balancer provider.
- **DNS:** AWS Route53 with ExternalDNS would be replaced by an on-prem DNS solution (e.g. BIND, Infoblox, or Active Directory DNS). ExternalDNS supports multiple providers including RFC2136 (dynamic DNS updates), Infoblox, and others. Alternatively, DNS records can be created manually or via existing automation.
- **Storage:** AWS `gp3-csi` StorageClass would be replaced by the on-prem storage provider (e.g. OpenShift Data Foundation / Ceph, NetApp Trident, Dell CSI, VMware vSAN).
- **Network latency:** Cross-region AWS latency (50-250ms) differs from typical inter-DC latency on a corporate WAN. Lower latency on-prem generally improves replica set performance. Higher latency may require tuning the `electionTimeoutMillis` and `heartbeatIntervalMillis` settings.
- **Firewall rules:** AWS security groups would be replaced by corporate firewall rules. The same ports must be open (see Network Requirements below).

The core architecture, operator configuration, and MongoDB replica set topology remain identical regardless of the hosting environment. All manifests in the repository use placeholder values that must be customised for the target environment.

Architecture

The solution deploys a 5-member MongoDB Enterprise replica set across 3 independent OpenShift clusters in a **2+2+1** configuration:



Failover behaviour: If DC1 is lost, the 2 members on DC2 plus the arbiter on Cluster 3 form a 3-of-5 majority. A DC2 member is elected primary within approximately 10-15 seconds. Writes resume automatically with no manual intervention. When DC1 recovers, its members rejoin as secondaries and catch up via the oplog.

Components

Component	Version	Purpose	Reference
MongoDB Controllers for Kubernetes (MCK)	1.8.0	Kubernetes operator managing MongoDB lifecycle	MCK Documentation
MongoDB Enterprise Server	8.0.4	Database engine	MongoDB Manual
MongoDB Ops Manager	8.0.21	Centralised management, monitoring, backup, automation	Ops Manager Documentation
ExternalDNS	0.16.1	Automatic DNS record creation for LoadBalancer services	ExternalDNS
cert-manager	latest (via OLM)	TLS certificate lifecycle management	cert-manager

---

Technical Decisions

1. Helm installation instead of OLM for the MCK operator

Decision: Install the MCK operator via Helm rather than the Operator Lifecycle Manager (OLM) / OperatorHub.

Reason: The multi-cluster variant of the MCK operator requires configuration parameters that OLM does not expose: - `multiCluster.clusters` - the list of member clusters must be provided at install time - `createOperatorServiceAccount=false` / `createResourcesServiceAccountsAndRoles=false` - required because the `kubectl mongodb multicluster setup` command creates these separately - `operator.watchNamespace` - must be scoped to specific namespaces rather than watching all namespaces

The certified-operators catalogue on OperatorHub ships only the single-cluster MCK operator. The multi-cluster variant is distributed exclusively via MongoDB's Helm chart.

Alternative: For a single-cluster MongoDB deployment (not spanning multiple clusters), OLM installation via OperatorHub is the standard and recommended approach. The OLM-based operator handles single-cluster replica sets, Ops Manager, and Community editions without any of the multi-cluster limitations.

Reference: Deploy the Operator in Multi-Kubernetes-Cluster Mode

2. No service mesh required

Decision: Deploy without Istio or any service mesh, using LoadBalancer services + ExternalDNS for cross-cluster connectivity.

Reason: MCK 1.8.0 introduced multi-cluster support without requiring a service mesh. Previous versions (pre-1.1.0) required Istio for cross-cluster communication. The no-mesh approach is simpler to operate and avoids the overhead of a full service mesh deployment.

How it works: Each MongoDB pod gets a dedicated LoadBalancer service. ExternalDNS creates DNS records mapping each pod's hostname to its LoadBalancer address. The `externalDomain` field in the `MongoDBMultiCluster` CR tells the operator to use these DNS names in the replica set configuration instead of cluster-local service names.

Alternative: If a service mesh (Istio, OpenShift Service Mesh) is already deployed, it can be used instead. This would eliminate the need for per-pod LoadBalancer services and ExternalDNS, but adds significant operational complexity.

Reference: Multi-Cluster Quick Start Without a Service Mesh

3. MongoDB Enterprise with Ops Manager (not Community edition)

Decision: Use MongoDB Enterprise Server managed by Ops Manager rather than MongoDB Community edition.

Reason: The `MongoDBMultiCluster` Custom Resource, which manages replica sets spanning multiple Kubernetes clusters, is only available with the Enterprise operator and requires Ops Manager. The Community edition operator (`MongoDBCommunity` CR) supports replica sets within a single cluster only.

What Ops Manager provides: - Centralised automation agent management across all clusters - Automated backup and point-in-time restore - Performance monitoring and alerting - Automated security (SCRAM, x.509, LDAP, Kerberos) - Rolling upgrades with zero downtime

Alternative: For single-cluster HA (3-member replica set within one OpenShift cluster), MongoDB Community edition is sufficient and does not require Ops Manager or an Enterprise licence. A separate Community edition deployment guide is available.

Reference: MongoDB Ops Manager Overview

4. SCRAM-SHA-256 authentication (TLS partially enabled)

Decision: Enable SCRAM-SHA-256 authentication on the data replica set. TLS is enabled on the Ops Manager Application Database but not on the data replica set API in this PoC.

Reason: During testing, the MCK operator could not verify self-signed TLS certificates when making bootstrap API calls to Ops Manager. Disabling TLS on the Ops Manager API was the workaround. The AppDB retains TLS.

For production: TLS should be fully enabled across all components. This requires either: - Using certificates signed by a trusted CA (not self-signed) - Configuring the operator to trust the self-signed CA (documented in MCK but encountered issues in 1.8.0)

Reference: Configure TLS for MongoDB Resources

5. GitOps-ready structure (Kustomize + Argo CD)

Decision: Structure the repository as Kustomize base + per-cluster overlays consumable by Argo CD ApplicationSets.

Reason: The expected production environment is GitOps-driven. The repository structure allows a single `ApplicationSet` to generate one Argo CD Application per cluster, with sync waves controlling deployment ordering.

Alternative: The repository also includes Ansible roles and playbooks for environments that do not use GitOps. Both approaches use the same underlying manifests.

Reference: Argo CD ApplicationSet

Testing and Outcomes

All tests were conducted on 19 May 2026 across three AWS-hosted OpenShift 4.20 clusters.

Test 1: Multi-cluster replica set formation

Objective: Verify that a 5-member replica set can form across 3 independent OpenShift clusters with no service mesh.

Procedure: 1. Deployed MCK operator on the central cluster via Helm 2. Ran `kubectl mongodb multicluster setup` to establish cross-cluster RBAC 3. Deployed Ops Manager on Cluster 1 4. Deployed ExternalDNS on all 3 clusters with per-cluster Route53 zones 5. Applied the `MongoDBMultiCluster` CR with `externalDomain` per cluster

Result: PASS - All 5 pods reached Running state across 3 clusters - DNS records created automatically by ExternalDNS (verified via Route53) - All 5 automation agents reached goal state in Ops Manager - Replica set formed with correct member configuration: - 1 PRIMARY on Cluster 1 (priority 10) - 1 SECONDARY on Cluster 1 (priority 10) - 2 SECONDARY on Cluster 2 (priority 5) - 1 SECONDARY on Cluster 3 (priority 0, arbiter role)

Test 2: Data replication verification

Objective: Verify that data written to the primary is replicated to all secondaries across clusters.

Procedure: 1. Connected to the primary on Cluster 1 2. Inserted 100 test documents into `testdb.failovertest` 3. Verified document count from secondaries on Cluster 2

Result: PASS - All 100 documents replicated to all secondaries - Verified via `db.failovertest.countDocuments()` on Cluster 2

Test 3: Cross-site failover

Objective: Simulate loss of DC1 (Cluster 1) and verify automatic failover to DC2 (Cluster 2).

Procedure: 1. Froze both Cluster 1 members using `db.adminCommand({replSetFreeze: 120})` to prevent re-election 2. Stepped down the primary using `rs.stepDown(120, 10)` 3. Monitored election on Cluster 2

Result: PASS - Cluster 2 member (`mongodb-rs-1-0`) elected as new PRIMARY within approximately 10 seconds - Both Cluster 1 members transitioned to SECONDARY state - Cluster 3 arbiter remained as SECONDARY (expected, priority 0)

Test 4: Write continuity after failover

Objective: Verify that writes succeed on the new primary after failover.

Procedure: 1. After failover, connected to the new primary on Cluster 2 2. Verified pre-failover data was intact (100 documents) 3. Inserted 10 additional documents 4. Verified total document count

Result: PASS - Pre-failover data intact: 100 documents - Post-failover writes succeeded: 10 new documents inserted - Total: 110 documents confirmed - Write continuity maintained with no data loss

---

Client Connection Options

Applications hosted on the same OpenShift cluster

Applications running on the same cluster as a MongoDB member can connect via the cluster-internal headless service:

```
mongodb://<user>:<password>@mongodb-rs-0-0-svc.mongodb-
data.svc.cluster.local:27017,mongodb-rs-0-1-svc.mongodb-
data.svc.cluster.local:27017/?replicaSet=mongodb-rs
```

This uses cluster-internal DNS and does not traverse the load balancer. Reads from local secondaries can be configured via read preferences (e.g. `readPreference=secondaryPreferred&readPreferenceTags=dc:cluster1`).

Applications hosted on a different cluster (but within the same network)

Applications on a different OpenShift cluster (or any host with network access to the LoadBalancer IPs) connect using the external DNS hostnames:

```
mongodb://<user>:<password>@mongodb-rs-0-0.<cluster1-
domain>:27017,mongodb-rs-0-1.<cluster1-domain>:27017,mongodb-
rs-1-0.<cluster2-domain>:27017,mongodb-rs-1-1.<cluster2-
domain>:27017,mongodb-rs-2-0.<cluster3-domain>:27017/?replicaSet=mongodb-
rs
```

All 5 members should be listed in the connection string. The MongoDB driver will automatically discover the current primary and route writes accordingly. If a failover occurs, the driver reconnects to the new primary transparently.

Applications outside the cluster network

For clients outside the cluster network (e.g. corporate workstations, CI/CD pipelines):

1. **Network path:** The client must be able to reach the LoadBalancer IPs (or DNS names) on port 27017. This may require VPN access or firewall rules.
2. **DNS resolution:** The external DNS hostnames must resolve from the client's network. If using split-horizon DNS, ensure the external names resolve to the correct LoadBalancer addresses.
3. **Authentication:** SCRAM-SHA-256 credentials are required. Connection strings must include `authSource=admin`.
4. **TLS (production):** When TLS is enabled, clients must trust the CA certificate used by the MongoDB deployment. Add the CA cert to the connection string: `tls=true&tlsCAFile=/path/to/ca.pem`.

Reference: MongoDB Connection String URI Format

Simplified connection via DNS SRV records (mongodb+srv)

Instead of listing every member hostname in the connection string, MongoDB supports `mongodb+srv://` connection strings backed by DNS SRV records. With this approach, clients connect using a single hostname:

```
mongodb+srv://<user>:<password>@mongodb-rs.example.com/?authSource=admin
```

The MongoDB driver performs an SRV lookup on `_mongodb._tcp.mongodb-rs.example.com` to discover all replica set members, and a TXT lookup for default connection options (e.g. `replicaSet=mongodb-rs&authSource=admin`).

To enable this, create the following DNS records in the shared domain:

Type	Name	Value
SRV	`_mongodb._tcp.mongodb-rs.example.com`	`0 0 27017 mongodb-rs-0-0.<cluster1-domain>`
SRV	`_mongodb._tcp.mongodb-rs.example.com`	`0 0 27017 mongodb-rs-0-1.<cluster1-domain>`
SRV	`_mongodb._tcp.mongodb-rs.example.com`	`0 0 27017 mongodb-rs-1-0.<cluster2-domain>`
SRV	`_mongodb._tcp.mongodb-rs.example.com`	`0 0 27017 mongodb-rs-1-1.<cluster2-domain>`
SRV	`_mongodb._tcp.mongodb-rs.example.com`	`0 0 27017 mongodb-rs-2-0.<cluster3-domain>`
TXT	`mongodb-rs.example.com`	`replicaSet=mongodb-rs&authSource=admin`

Benefits: - Client connection strings never change, even if members are added, removed, or migrate between clusters - Failover is completely transparent to clients - Application configuration is simplified (one hostname instead of five) - TLS and read preference options can be set centrally via the TXT record

Note: The SRV records can be created in any DNS zone accessible to the clients. They do not need to be in the same zones as the per-pod A records. This makes it straightforward to present a single stable entry point (e.g. `mongodb-rs.corp.example.com`) regardless of which clusters host the members.

Reference: DNS Seed List Connection Format

Read preference for geographic locality

Applications can use read preference tags to route reads to the nearest data centre. This works with both `mongodb://` and `mongodb+srv://` connection strings:

```
mongodb+srv://<user>:<password>@mongodb-rs.example.com/?
readPreference=secondaryPreferred&readPreferenceTags=dc:cluster1&readPref
erenceTags=dc:cluster2
```

This reads from DC1 secondaries first, falls back to DC2, and only reads from the primary if no secondaries are available.

Reference: Read Preference

Prerequisites

Infrastructure

Requirement	Detail
OpenShift clusters	3 clusters running OpenShift 4.14 or later, each with at least 2 worker nodes
Worker node sizing	Minimum 16 GB RAM, 4 vCPU per worker node (MongoDB Ops Manager is memory-intensive)
Storage	A StorageClass supporting `ReadWriteOnce` dynamic provisioning on each cluster (e.g. ODF/Ceph, NetApp Trident, VMware vSAN, AWS gp3-csi)
DNS	A DNS zone per cluster where ExternalDNS (or manual processes) can create A/CNAME records. All clusters must be able to resolve all zones.
Load balancer	A load balancer provider per cluster capable of provisioning per-service external IPs (e.g. MetalLB, F5 BIG-IP, AWS NLB)
Network connectivity	TCP port 27017 open between all 3 clusters (bidirectional). TCP port 8080/8443 from all clusters to the Ops Manager cluster.

Software

Requirement	Detail
MongoDB Enterprise licence	Required for Ops Manager and the Enterprise server binaries. Contact MongoDB sales or use an existing Enterprise Advanced subscription.
MCK operator Helm chart	Version 1.8.0 from the MongoDB Helm repository
`kubectl-mongodb` plugin	Multi-cluster setup tool. Download from MCK 1.8.0 releases
cert-manager	For TLS certificate management. Available via OLM (Red Hat certified operator) or Helm.

Requirement	Detail
ExternalDNS	For automatic DNS record creation. Deployed as a standard Kubernetes Deployment.
Argo CD / OpenShift GitOps	For GitOps-driven deployment (optional, manual `kustomize build | kubectl apply` also supported)

Access and credentials

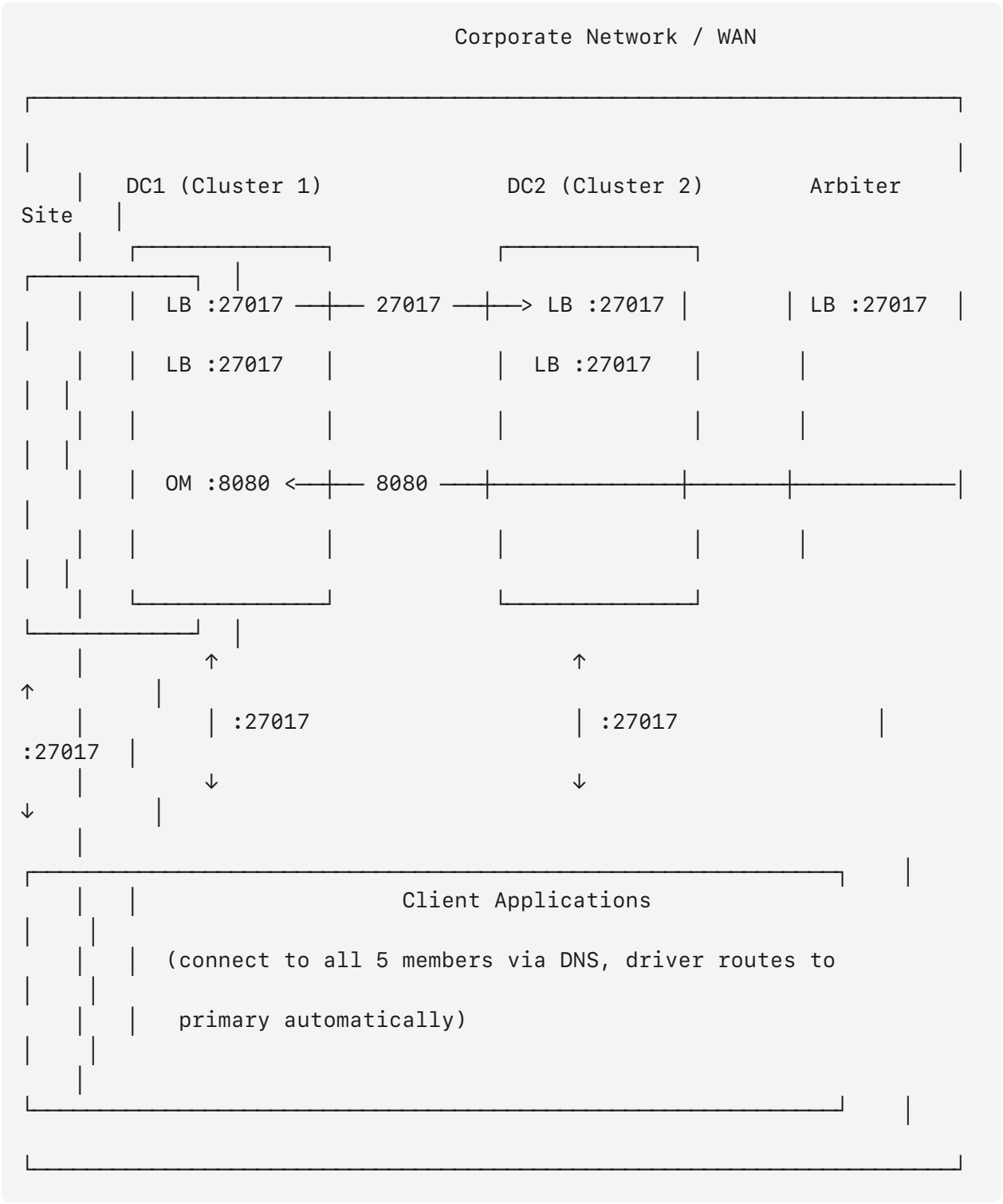
Requirement	Detail
`kubeadmin` or cluster-admin	Required on all 3 clusters for initial setup (SCC grants, CRD installation, operator deployment)
DNS zone credentials	Credentials for the DNS provider to allow ExternalDNS to create records (e.g. cloud provider API keys, RFC2136 TSIG keys)
MongoDB Ops Manager	After initial deployment, a programmatic API key with Project Owner permissions must be created via the Ops Manager UI

Network Requirements

Port matrix

Source	Destination	Port	Protocol	Purpose
MongoDB pods (all clusters)	MongoDB pods (all clusters)	27017/ TCP	MongoDB wire protocol	Replica set communication (heartbeats, oplog sync, elections)
MongoDB pods (all clusters)	Ops Manager (Cluster 1)	8080/ TCP	HTTP	Automation agent communication, binary downloads
Ops Manager (Cluster 1)	MongoDB pods (all clusters)	27017/ TCP	MongoDB wire protocol	Monitoring, backup agent connections
External clients	MongoDB LoadBalancer IPs	27017/ TCP	MongoDB wire protocol	Client application connections
ExternalDNS pods	DNS provider API	443/ TCP	HTTPS	DNS record creation/updates (e.g. Route53 API, Infoblox API)

Network flow diagram



All cross-cluster traffic flows over the corporate WAN via LoadBalancer external IPs. No cluster-internal (ClusterIP/pod network) traffic crosses cluster boundaries.

Ops Manager Deployment: Current State and Production Recommendation

Current PoC state

In this proof of concept, Ops Manager runs as a **single instance on Cluster 1 only**. This was a deliberate simplification to focus on validating the cross-cluster data replica set. The Ops Manager multi-cluster topology (active/standby with distributed AppDB) was planned but not fully validated due to issues encountered with cross-cluster TLS trust during operator bootstrap.

Impact if Cluster 1 is lost

The MongoDB data replica set **continues to operate normally** if Cluster 1 goes down. Failover to Cluster 2 proceeds as tested. However, Ops Manager itself becomes unavailable until Cluster 1 recovers. During that time:

- The data replica set keeps running independently (it does not depend on Ops Manager at runtime)
- Monitoring dashboards and alerting are paused
- Backup scheduling and snapshot creation are paused
- No new deployments or automation config changes can be made

Data availability is not affected. Only management-plane operations are impacted.

Production recommendation

For production, Ops Manager should be deployed in multi-cluster mode:

- **Active instance** on DC1, **standby instance** on DC2 (automatic promotion if DC1 is lost)
- **Application Database** spread 1+1+1 across all three sites
- **Backup daemons** on both DC1 and DC2

This ensures the management plane remains available even during a full site failure. The MCK operator supports this topology via the `MongoDBOpsManager` CR with `topology: MultiCluster`. The Ansible roles in the repository already contain templates for this configuration, though it was not end-to-end validated in the PoC.

References: - Ops Manager Resource Specification - Deploy Ops Manager in Multi-Cluster Mode

---

Known Limitations and Workarounds

These were discovered during the PoC and are documented as issues to be aware of during production deployment.

#	Issue	Workaround	Status
1	`kubectl mongodb multicluster setup` does not grant `mongodb.com` API group permissions on member clusters, causing `blockOwnerDeletion` errors	Manually patch the Role on member clusters with the missing permissions	Bootstrap script provided (`05-patch-member-roles.sh`)
2	CRDs are not installed on member clusters by the setup command	Copy CRDs from the central cluster to member clusters	Bootstrap script provided (`06-install-crds-members.sh`)
3	MCK 1.8.0 pushes `svc.cluster.local` hostnames to the Ops Manager automation config even when `externalDomain` is set in the CR	Manually update the automation config via the Ops Manager API after initial deployment	Bootstrap script provided (`07-fix-om-hostnames.sh`). The operator adopts the corrected hostnames on subsequent reconciliations.
4	Ops Manager bootstrap fails with self-signed TLS certificates ("certificate signed by unknown authority")	Disable TLS on the Ops Manager API. For production, use a trusted CA.	Documented in the Ops Manager CR
5	OpenShift `restricted` SCC blocks MongoDB pods (they require specific UIDs)	Grant `anyuid` SCC to MongoDB ServiceAccounts	Bootstrap script provided (`04-grant-scc.sh`). OpenShift SCC docs
6	Agents on member clusters cannot reach Ops Manager via cluster-local service names	Expose Ops Manager via an OpenShift Route; configure agents to use the external URL	Route manifest and ConfigMap included
7	Multi-cluster MCK operator not available via OLM/OperatorHub	Install via Helm	Bootstrap script provided (`02-helm-install-mck-operator.sh`)

References: - MCK Multi-Cluster Troubleshooting - MCK Known Issues

Repository and Source Code

The complete source code, Kustomize manifests, bootstrap scripts, and Ansible playbooks are available at:

https://github.com/amasolov/mongodb-multicluster-ocp

Further Reading

- MongoDB Controllers for Kubernetes Documentation
- Multi-Cluster Replica Set Architecture
- Deploy Multi-Cluster Without a Service Mesh
- Configure ExternalDNS for Multi-Cluster
- MongoDB Ops Manager Architecture
- Ops Manager API Reference
- Ops Manager Backup Overview
- Geographically Distributed Replica Sets
- Replica Set Elections
- MongoDB Connection String URI Format
- Read Preference Configuration
- Production Notes for MongoDB